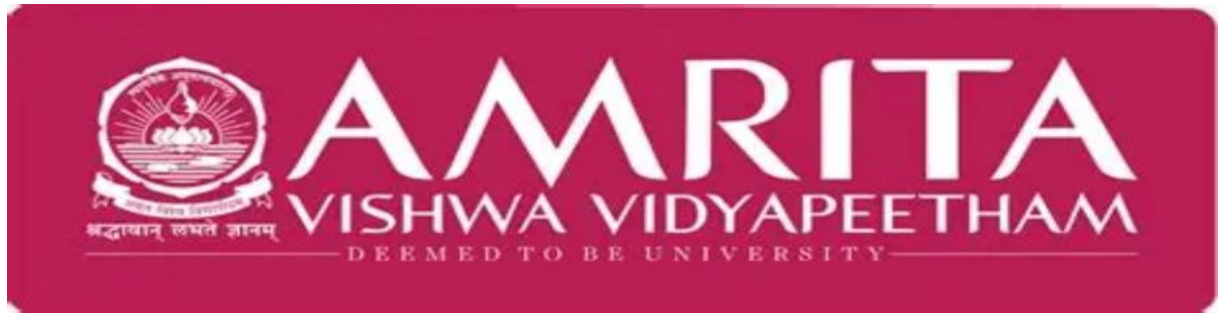




DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	NADIPALLI HASINI
Roll No	AV.SC.U4CSE25133
Year/Sem/Section	2nd year/Sem-3/CSE-B
Credits	05

Submitted to	
Name	Dr. RAJ KUMAR BATCHU
Department	School of Computing
Designation	Asst.Professor(SL Grade)

INDEX

Sno	Program	Page No.	Remarks
1.	WEEK 1: A. A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”. B. The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p=principal growth factor, n= number of years. C. An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. D. In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number. E. Write a recursive python program to print the first n terms of the Fibonacci series.	5-14	
2.	WEEK 2: A. Write a program to perform linear search on a given input. B. (i) Write a python program to implement binary search with sorted array as an input. (ii) Write a python program to implement binary search without sorted array as an input	15-22	
3.	WEEK 3:	23-28	

	A. Write a python program to implement bubble sort, where the input elements are taken from the user. B. Write a python program to implement insertion sort, where the input elements are taken from the user. C. Write a python program to implement selection sort, where the input elements are taken from the user.		
4.	WEEK 4: A. Write a python program to implement Merge sort, where the input elements are taken from the user. B. Write a python program to implement Quick sort, where the input elements are taken from the user.	29-33	
5.	WEEK 5: A. Write a Python program to implement a singly linked list with the below operations: 1. Create linked list 2. Insert at beginning 3. Insert at end 4. Insert at a specific index 5. Delete by value 6. Delete first node 7. Delete last node 8. Count number of nodes 9. Display / Traverse 10. Exit B. Execute above programs for doubly linked list C. Execute above programs for Circular linked list	34-58	

6.	WEEK 6:	59-66	
----	---------	-------	--

	A. Write a Python program to implement stack operations such as Push, Pop, Peek, Display using an array B. Write a Python program to implement stack operations such as Push, Pop, Peek, Display using an Linked lists		
7.	WEEK 7: A. Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using arrays B. Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using linked list data structures. C. Circular Queue- its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using arrays. D. Circular Queue- its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using linked-list data structures.	67-82	

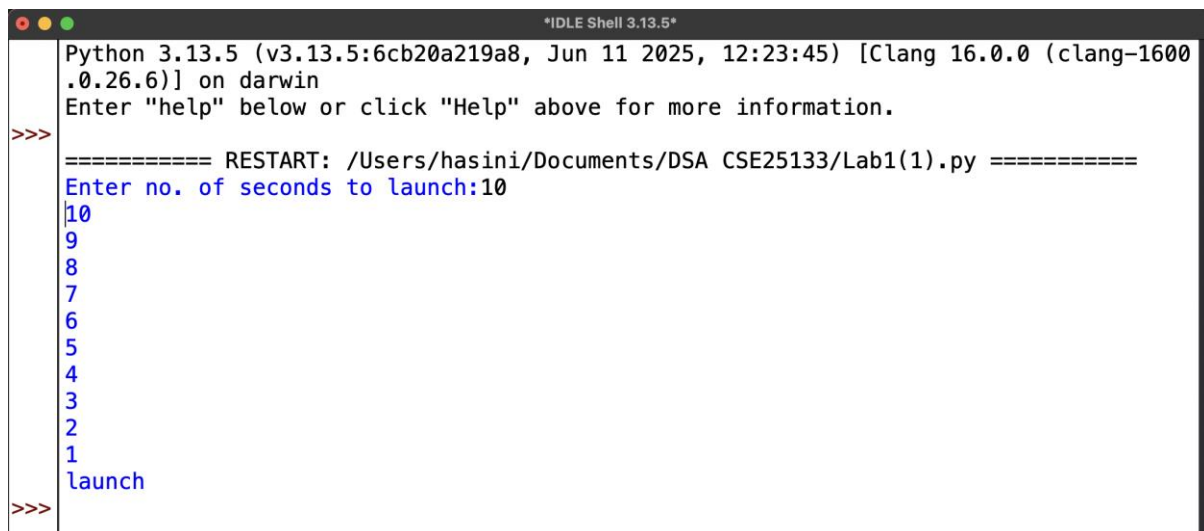
Week-1

A.AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):  
    for i in range (n, -1,-1):  
        if i==0:  
            print("launch")  
        else:  
            print(i)  
  
n=int (input ("Enter no. of seconds to launch:"))  
  
launch(n)
```

OUTPUT:



```
*IDLE Shell 3.13.5*  
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600  
.0.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(1).py =====  
Enter no. of seconds to launch:10  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
launch  
>>>
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 4 spaces for each level of indentation.
2.	Type Error	Convert the input to an integer using <code>n = int (input ("Enter no. of seconds to launch: "))</code> .

B.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p =principal growth factor, n = number of years.

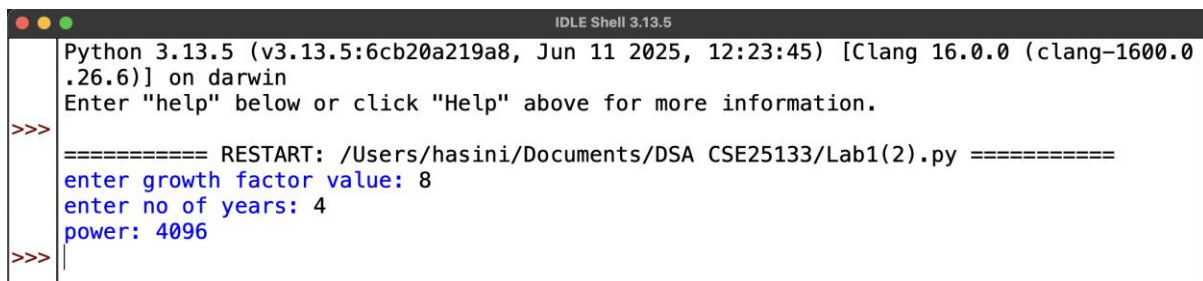
PROGRAM:

```
p = float (input ("Enter principal growth factor (p): "))
n = int (input ("Enter number of years (n): "))

def power (p, n):
    if n == 0:
        return 1
    else:
        return p * power (p, n - 1)

result = power (p, n)
print ("Power: ", result)
```

OUTPUT:

A screenshot of a Python IDLE Shell window titled "IDLE Shell 3.13.5". The window shows the following text: "Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin", "Enter 'help' below or click 'Help' above for more information.", and a prompt ">>>". Below the prompt, there is a line of text "===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(2).py =====". Then, the user input "enter growth factor value: 8" is shown, followed by "enter no of years: 4", and the output "power: 4096" is displayed. The prompt ">>>" is shown again at the bottom.

EXPLANATION:

- The function of power (p , n) calculates p^n using recursion by multiplying p with the result of power (p , $n-1$).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

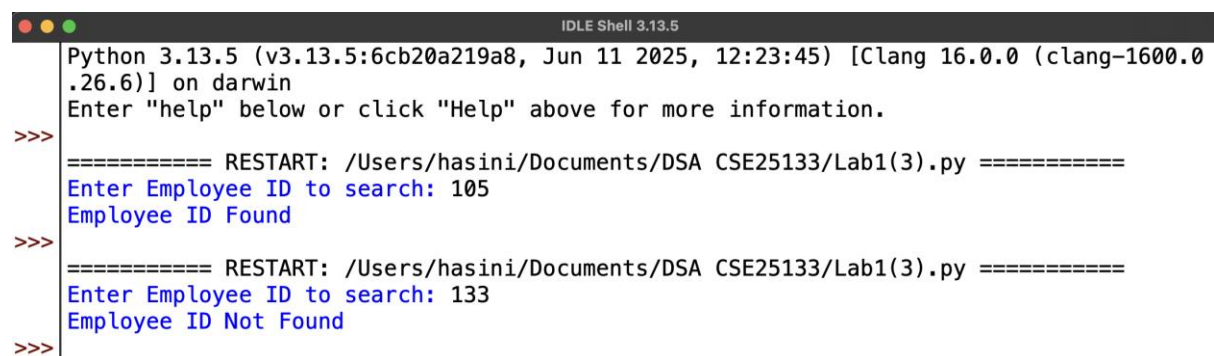
S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float (input()) or int (input ()).
2.	Incorrect output if the recursive call is written as power (p, n + 1) instead of power (p, n - 1).	Change the recursive call to power (p, n - 1) so that n decreases toward the base case.

C.AIM: An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):  
    if index >= len(list):  
        return False  
    if list[index] == key:  
        return True  
    return search (lst, key, index + 1)  
employees = [101, 102, 103, 104, 105]  
key = int (input ("Enter Employee ID to search: "))  
if search (employees, key, 0):  
    print ("Employee ID Found")  
else:  
    print ("Employee ID Not Found")
```

OUTPUT:



```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>>  
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(3).py =====  
Enter Employee ID to search: 105  
Employee ID Found  
>>>  
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(3).py =====  
Enter Employee ID to search: 133  
Employee ID Not Found  
>>>
```

EXPLANATION:

- The recursive function search () checks each employee ID in the list one by one, starting from the given index.
- If the current employee ID matches the required ID (key), it returns True.
- If the ID does not match, the function calls itself recursively with the next index (index + 1).

- If the index reaches the end of the list, it returns False, indicating that the employee ID was not found.
- Finally, the if-else statement displays whether the employee ID is **Found** or **Not Found**.

ERROR TABLE:

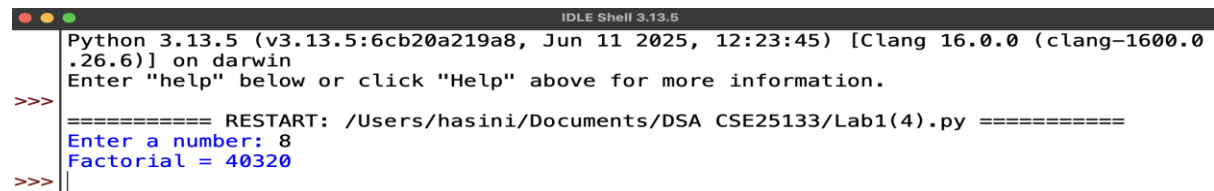
S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., lst instead of list).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

D.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial (n - 1)  
  
num = int (input ("Enter a number: "))  
  
if num < 0:  
    print ("Factorial does not exist for negative numbers.")  
else:  
    print ("Factorial =", factorial(num))
```

OUTPUT:



```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0  
.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(4).py =====  
Enter a number: 8  
Factorial = 40320  
>>>
```

EXPLANATION:

- The function `factorial(n)` computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1 , where the factorial value is 1 .
- The program also checks for negative inputs since factorials are defined only for non-negative integers.

ERROR TABLE:

S.no	Errors	Rectification
1.	Name Error occurs when a variable or function is used without defining it first.	Define the variable or function before using it in the program.
2.	Value Error occurs when an invalid value, such as a letter, is entered where a number is expected.	Enter a valid numerical value or use proper input validation.

E.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
n = int (input ("enter number of terms: "))

def fibonacci(num):

    if num==0:

        return 0

    elif num==1:

        return 1

    else:

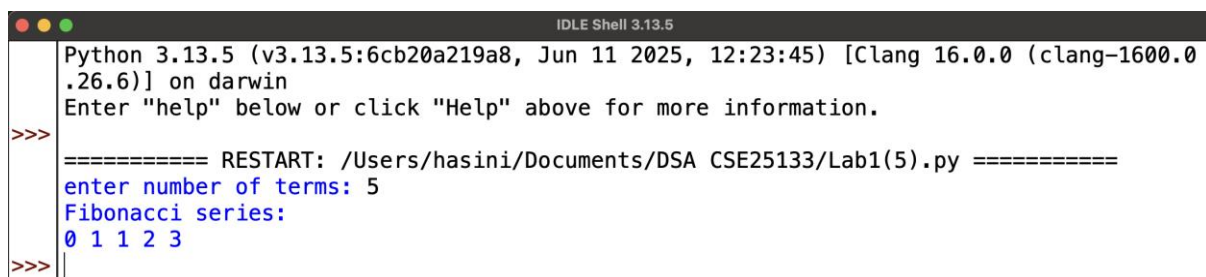
        return fibonacci(num-1) +fibonacci(num-2)

print ("Fibonacci series: ")

for i in range(n):

    print(fibonacci(i), end=" ")
```

OUTPUT:

A screenshot of a Python IDLE Shell window. The title bar reads "IDLE Shell 3.13.5". The window content shows the Python version and environment: "Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin". It prompts the user to enter "help" for more information. The user has entered "5" in response to the prompt "enter number of terms: 5". The program has executed, printing "Fibonacci series:" followed by the numbers "0 1 1 2 3" on the same line. The prompt ">>>" is visible at the end of the line.

```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(5).py =====
>>> enter number of terms: 5
>>> Fibonacci series:
>>> 0 1 1 2 3
>>>
```

EXPLANATION:

- The function `Fibonacci(n)` returns the n th Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return `0` for the first term and `1` for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

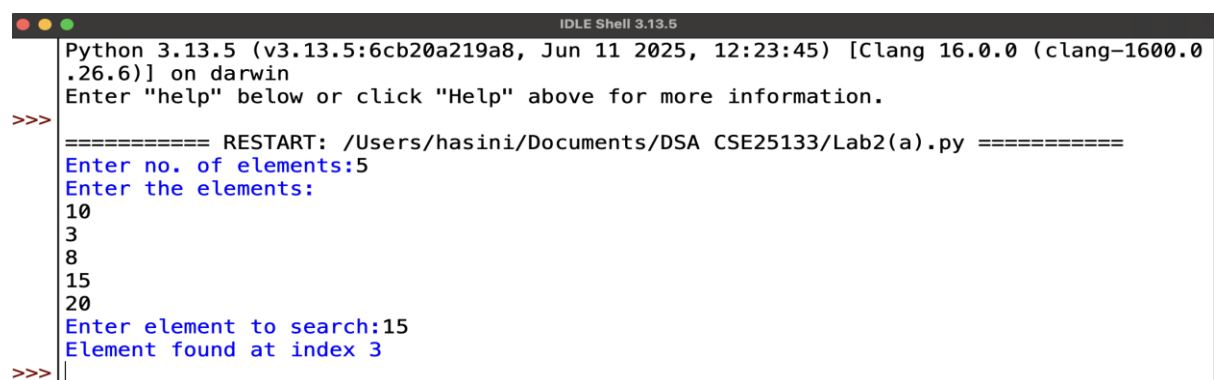
Week-2

A.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):  
    for i in range(len(arr)):  
        if arr[i]==key:  
            return i  
    return -1  
  
n = int (input ("Enter no. of elements:"))  
arr= []  
print ("Enter the elements:")  
for i in range(n):  
    arr.append(int(input ()))  
key=int (input ("Enter element to search:"))  
result=linear_search(arr,key)  
if result!= -1:  
    print ("Element found at index", result)  
else:  
    print ("Element not found.")
```

OUTPUT:



```
IDLE Shell 3.13.5  
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0  
.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(a).py =====  
Enter no. of elements:5  
Enter the elements:  
10  
3  
8  
15  
20  
Enter element to search:15  
Element found at index 3  
>>>
```

EXPLANATION:

- The linear_search() function checks each element of the list one by one and compares it with the given key.
- If the key is found, its index is returned; otherwise, the function returns - 1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int (input ()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

B(i). AIM: Write a python program to implement binary search with sorted array as an input.

PROGRAM:

```
def binary_search(arr, key):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1  
  
n = int (input ("Enter number of elements: "))  
  
arr = []  
  
print ("Enter elements in sorted order:")  
  
for i in range(n):  
    arr.append(int(input ()))  
  
key = int (input ("Enter the element to search: "))  
  
result = binary_search(arr, key)  
  
if result != -1:  
    print("Element found at index", result)  
else:
```

```
print("Element not found")
```

OUTPUT:

```
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(i)).py =====
Enter number of elements: 5
Enter elements in sorted order:
11
22
33
44
55
Enter the element to search: 44
Element found at index 3
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(i)).py =====
Enter number of elements: 5
Enter elements in sorted order:
11
22
33
44
55
Enter the element to search: 66
Element not found
>>>
```

EXPLANATION:

- The program defines a `binary_search()` function that takes a sorted list and a search element as input.
- The variables `low` and `high` are initialized to represent the first and last positions of the list.
- The program calculates the middle position using `mid = (low + high) // 2` and compares the middle element with the search element.
- If the search element is greater or smaller than the middle element, the program searches only the required half of the list.
- If the element is found, its index is returned; otherwise, `-1` is returned and **"Element not found"** is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error if the statements inside the function or loop are not properly indented.	Maintain proper indentation inside the function and loop.
2.	ValueError if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
3.	Incorrect result if the elements are not entered in sorted order.	Enter the elements in sorted order before performing binary search.
4.	IndexError if an invalid index is accessed while searching.	Make sure low and high remain within the list range.

B(ii). AIM: Write a python program to implement binary search without sorted(unsorted) array as an input

PROGRAM:

```
def binary_search(arr, key):  
    arr.sort()  
  
    low = 0  
  
    high = len(arr) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
  
        if arr[mid] == key:  
            return mid  
  
        elif arr[mid] < key:  
            low = mid + 1  
  
        else:  
            high = mid - 1  
  
    return -1  
  
n = int (input ("Enter number of elements: "))  
  
arr = []  
  
print ("Enter elements:")  
  
for i in range(n):  
    arr.append(int(input()))  
  
key = int (input ("Enter the element to search: "))  
  
result = binary_search(arr, key)  
  
if result != -1:  
    print ("Element found at index", result)
```

else:

print ("Element not found")

OUTPUT:

```
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(ii)).py =====
Enter number of elements: 5
Enter elements in unsorted order:
45
33
2
5
18
Enter the element to search: 33
Element found at index 3
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(ii)).py =====
Enter number of elements: 5
Enter elements in unsorted order:
45
33
2
5
18
Enter the element to search: 55
Element not found
>>>
```

EXPLANATION:

1. The `binary_search()` function is defined to search for the given key in the list `arr`.
2. The `arr.sort()` function arranges the elements in ascending order so that binary search can be performed.
3. The variables `low` and `high` represent the first and last positions of the sorted list.
4. The program finds the middle element and compares it with the key. Based on the comparison, it searches either the left half or the right half.
5. If the element is found, its index is returned. If the element is not found, the function returns `-1`.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError may occur if while, if, or other statements are not properly aligned.	Maintain proper indentation inside the function and conditional blocks.
2.	SyntaxError may occur if – is used instead of the normal minus operator - in high = mid – 1.	Use the normal hyphen: high = mid - 1.
3.	ValueError occurs if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
4.	NameError occurs if variables such as arr, key, or result are misspelled.	Check the spelling and define variables before using them.

Week-3

A.AIM: Write a python program to implement bubble sort, where the input elements are taken from the user.

PROGRAM:

```
#bubble sort

def bubble_sort(arr):

    n = len(arr)

    for i in range (n-1):

        for j in range(n-i-1):

            if arr[j]>arr[j+1]:

                arr[j],arr[j+1] = arr[j+1],arr[j]

    return arr

#taking input

n = int(input("Enter number of elements: "))

arr=[]

print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

bubble_sort(arr)

print("sorted array: ")

for element in arr:

    print(element, end=" ")
```

OUTPUT:

```

>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(1) bubble sort.py =====
Enter number of elements: 5
Enter elements:
12
65
33
25
1
sorted array:
1 12 25 33 65
>>>

```

EXPLANATION:

- The `bubble_sort()` function is defined to arrange the elements of the list in ascending order.
- The outer for loop controls the number of passes, while the inner for loop compares adjacent elements.
- The condition `arr[j] > arr[j+1]` checks whether the current element is greater than the next element.
- If the elements are in the wrong order, they are exchanged using `arr[j], arr[j+1] = arr[j+1], arr[j]`.
- After all comparisons and swaps are completed, the sorted array is returned and displayed element by element.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values and linely
2.	NameError if <code>arr</code> , <code>n</code> , or another variable is misspelled.	Check the variable names and use the correct spelling.
3.	TypeError may occur if the list contains incompatible data types during comparison.	Enter elements of the same suitable data type, such as integers.
4.	IndexError may occur if the program accesses an invalid array position.	Use the correct loop range <code>range(n-i-1)</code> to keep the index within the list.

B.AIM: Write a python program to implement insertion sort, where the input elements are taken from the user.

PROGRAM:

```
def insertion_sort(arr): n = len(arr) for i in range (1, n): key = arr[i] j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1
    arr[j + 1] = key
return arr
#Taking input

n = int (input ("Enter number of elements: ")) arr = []

print ("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

#Sorting the array

insertion_sort(arr)

print ("Sorted array: ")

for element in arr:

    print (element, end=" ")
```

OUTPUT:

```
==== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(2) insertion sort.py ====
Enter number of elements: 6
Enter elements:
42
14
35
54
33
29
Sorted array:
14 29 33 35 42 54
>>>
```

EXPLANATION:

- The `insertion_sort()` function is defined to arrange the elements of the list in **ascending order**.
- The for loop starts from the second element because the first element is considered already sorted.
- The variable `key` stores the current element, and `j` is used to compare it with the previous elements.
- The while loop shifts elements that are greater than `key` one position to the right.
- The `key` is placed in its correct position, and after all elements are processed, the sorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function, for, or while loop are not properly indented.	Maintain proper indentation inside all blocks.
2.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	NameError if variables such as <code>arr</code> , <code>key</code> , or <code>j</code> are misspelled.	Check the variable names and use the correct spelling.
4.	IndexError if an invalid list index is accessed while shifting elements.	Make sure <code>j >= 0</code> is checked before accessing <code>arr[j]</code> .

C.AIM: Write a python program to implement selection sort, where the input elements are taken from the user.

PROGRAM:

```
#Selection Sort

def selection_sort(arr): n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

#Taking input

n = int(input("Enter number of elements: "))

arr = [] print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

#Sorting the array

selection_sort(arr)

print("Sorted array: ")

for element in arr:

    print(element, end=" ")
```

OUTPUT:

```

>>>
==== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(3) selection sort.py ====
Enter number of elements: 6
Enter elements:
22
42
12
2
72
32
Sorted array:
2 12 22 32 42 72
>>>

```

EXPLANATION:

- The `selection_sort()` function is defined to arrange the elements of the list in **ascending order**.
- The outer for loop selects the position where the smallest element should be placed.
- The variable `min_index` stores the index of the smallest element found in the unsorted part of the list.
- The inner for loop compares the elements and updates `min_index` whenever a smaller element is found.
- The smallest element is swapped with the element at position `i`, and after all passes, the sorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the function or loop statements are not properly indented.	Maintain proper indentation inside the function and loops.
2.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	NameError if variables such as <code>arr</code> , <code>i</code> , <code>j</code> , or <code>min_index</code> are misspelled.	Check the variable names and use the correct spelling.
4.	IndexError if an invalid list index is accessed.	Use the correct loop ranges and ensure the index is within the list range.

Week-4

A.AIM: Write a python program to implement merge sort, where the input elements are taken from the user.

PROGRAM:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
    left = merge_sort(left)
    right = merge_sort(right)
    i = j = k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
```

```

        k += 1

    return arr

n = int(input("Enter number of elements: "))

arr = []

print("Enter elements:")

for i in range(n):

    arr.append(int(input()))

merge_sort(arr)

print("Sorted array:")

for element in arr:

    print(element, end=" ")

```

OUTPUT:

```

===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab4(a) merge sort.py =====
Enter number of elements: 5
Enter elements:
16
52
22
7
4
Sorted array:
4 7 16 22 52
>>>|

```

EXPLANATION:

- The merge_sort() function is defined to arrange the elements of the list in **ascending order** using the Merge Sort algorithm.
- The if len(arr) <= 1 condition checks whether the list has **one or no elements**. If so, the list is already sorted and returned.
- The variable mid stores the **middle index** of the list, which is used to divide the list into two halves.
- The list is divided into two parts: the left contains the first half and the right contains the second half.
- The merge_sort() function is called recursively to **sort both the left and right halves**.
- The variables i, j, and k are used as indexes for the **left list, right list, and original list** respectively.
- The while loop compares the elements of the left and right lists and places the **smaller element** into the original list.
- The next while loops copy any **remaining elements** from the left or right list into the original list.

- The return arr statement returns the **sorted list** after merging the two halves.
- The user enters the **number of elements** and the elements of the list using input().
- The merge_sort(arr) function is called to **sort the given list**.
- Finally, the sorted elements are displayed using a for loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because some statements inside the function and loops were not properly aligned.	Corrected the indentation of the statements according to Python syntax.
2.	ValueError occurred when I entered a non-numeric value while giving the array elements.	Entered only integer values as input for the array.
3.	TypeError occurred because incompatible data types were present while comparing the elements.	Used integer values of the same suitable data type for sorting.
4.	Recursion Error occurred because the recursive function did not reach the stopping condition properly.	Added and verified the base condition if len(arr) <= 1.
5.	IndexError occurred when the program tried to access an element outside the valid range of the list.	Corrected the loop conditions and ensured that the indexes remain within the list range.

B.AIM: Write a python program to implement quick sort, where the input elements are taken from the user.

PROGRAM:

```
def quicksort(a, low, high):  
    if low < high:  
        i = low  
        j = high  
        pivot = low  
        while i < j:  
            while i <= high and a[i] <= a[pivot]:  
                i += 1  
            while a[j] > a[pivot]:  
                j -= 1  
            if i < j:  
                a[i], a[j] = a[j], a[i]  
            a[pivot], a[j] = a[j], a[pivot]  
            quicksort(a, low, j - 1)  
            quicksort(a, j + 1, high)  
n = int(input("Enter number of elements: "))  
a = []  
print("Enter elements:")  
for i in range(n):  
    a.append(int(input()))  
quicksort(a, 0, n - 1)  
print("Sorted array:")  
for element in a:  
    print(element, end=" ")
```

OUTPUT:

```

===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab4(b) quicksort.py =====
Enter number of elements: 5
Enter elements:
25
12
81
18
11
Sorted array:
11 12 18 25 81

```

EXPLANATION:

- The quicksort() function is used to arrange the elements of the array in **ascending order**.
- The condition if low < high checks whether the array portion contains more than one element.
- The variables i and j are used to **scan the array from both sides**.
- The variable pivot stores the index of the **first element selected as the pivot**.
- The first while loop moves i forward until an element greater than the pivot is found.
- The second while loop moves j backward until an element smaller than or equal to the pivot is found.
- If $i < j$, the elements at i and j are **swapped**.
- The pivot is then swapped with the element at j, placing it in its **correct position**.
- The function is recursively called for the **left and right portions** of the pivot.
- The user enters the number of elements and their values, and quicksort() is called to sort the array.
- Finally, the **sorted array is displayed**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurred when I entered a non-integer value for an array element.	Entered only valid integer values while providing the input.
2.	NameError occurred because I used an incorrect variable name such as pivot, i, or j.	Checked the variable names and corrected their spelling throughout the program.
3.	Index Error occurred when the program tried to access an array position outside its valid range.	Corrected the loop conditions and ensured that the indexes remain within the array boundaries.
4.	Recursion Error occurred when the recursive calls continued without reaching the proper stopping condition.	Ensured that the condition if low < high correctly controls the recursive calls.

Week-5

A.AIM: Write a python program to implement single linked list with below operations

1. Create Linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit

PROGRAM:

```
class Node: def init(self, data): self.data = data self.next = None
```

```
class SinglyLinkedList: def init(self): self.head = None
```

```
# 1. Create linked list
```

```
def create(self):
```

```
    n = int(input("Enter number of nodes: "))
```

```
    for i in range(n):
```

```
        data = int(input(f"Enter data for node {i + 1}: "))
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next:
```

```
                temp = temp.next
```

```
            temp.next = new_node
```

```
    print("Linked list created successfully.")
```

```
# 2. Insert at beginning
```

```
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    new_node.next = self.head
    self.head = new_node

    print("Node inserted at beginning.")
```

3. Insert at end

```
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:

            temp = temp.next

        temp.next = new_node

    print("Node inserted at end.")
```

4. Insert at specific index

```
def insert_index(self):

    index = int(input("Enter index: "))

    data = int(input("Enter data: "))

    new_node = Node(data)

    if index == 0:

        new_node.next = self.head

        self.head = new_node

    print("Node inserted.")

    return
```

```

temp = self.head

for i in range(index - 1):

    if temp is None:

        print("Invalid index.")

        return

    temp = temp.next

if temp is None:

    print("Invalid index.")

    return

new_node.next = temp.next

temp.next = new_node

print("Node inserted.")

# 5. Delete by value

def delete_value(self):

    value = int(input("Enter value to delete: "))

    if self.head is None:

        print("List is empty.")

        return

    if self.head.data == value:

        self.head = self.head.next

        print("Node deleted.")

        return

    temp = self.head

    while temp.next and temp.next.data != value:

```

```
        temp = temp.next

    if temp.next is None:

        print("Value not found.")

    else:

        temp.next = temp.next.next

        print("Node deleted.")

# 6. Delete first node

def delete_first(self):

    if self.head is None:

        print("List is empty.")

    else:

        self.head = self.head.next

        print("First node deleted.")

# 7. Delete last node

def delete_last(self):

    if self.head is None:

        print("List is empty.")

        return

    if self.head.next is None:

        self.head = None

        print("Last node deleted.")

        return

    temp = self.head

    while temp.next.next:
```

```
        temp = temp.next

    temp.next = None

    print("Last node deleted.")

# 8. Count nodes

def count(self):

    count = 0

    temp = self.head

    while temp:

        count += 1

        temp = temp.next

    print("Number of nodes:", count)

# 9. Display / Traverse

def display(self):

    if self.head is None:

        print("List is empty.")

        return

    temp = self.head

    while temp:

        print(temp.data, end=" -> ")

        temp = temp.next

    print("None")

# Main program

sll = SinglyLinkedList()

while True:

    print("\n--- SINGLY LINKED LIST ---")
```

```
print("1. Create linked list")

print("2. Insert at beginning")

print("3. Insert at end")

print("4. Insert at specific index")

print("5. Delete by value")

print("6. Delete first node")

print("7. Delete last node")

print("8. Count number of nodes")

print("9. Display / Traverse")

print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:

    sll.create()

elif choice == 2:

    sll.insert_beginning()

elif choice == 3:

    sll.insert_end()

elif choice == 4:

    sll.insert_index()

elif choice == 5:

    sll.delete_value()

elif choice == 6:

    sll.delete_first()

elif choice == 7:

    sll.delete_last()
```

```

elif choice == 8:

    sll.count()

elif choice == 9:

    sll.display()

elif choice == 10:

    print("Program exited.")

    break

else:

    print("Invalid choice.")

```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab5 sll.py =====

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 11
Invalid choice.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 11
Enter data for node 2: 18
Enter data for node 3: 81
Enter data for node 4: 2
Enter data for node 5: 25
Linked list created successfully.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse

```

```

Enter your choice: 2
Enter data: 5
Node inserted at beginning.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 55
Node inserted at end.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 3
Enter data: 33
Node inserted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 100
Value not found.

```

```

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 5
Node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value

```

```

6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
18 -> 33 -> 81 -> 2 -> 25 -> None

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.

```

EXPLANATION:

- The Node class is used to create a **node containing data and a link to the next node**.
- The SinglyLinkedList class maintains the linked list using the head pointer.

- The create() function creates a linked list by adding the given number of nodes.
- The insert_beginning() function adds a **new node at the beginning** of the list.
- The insert_end() function adds a **new node at the end** of the list.
- The insert_index() function inserts a node at a **specified index**.
- The delete_value() function searches for a given value and **deletes the corresponding node**.
- The delete_first() function removes the **first node** from the list.
- The delete_last() function removes the **last node** from the list.
- The count() function counts and displays the **total number of nodes**.
- The display() function traverses the list and **prints all the elements**.
- The main program displays a **menu** and performs the selected linked-list operation.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Node Error occurred because the Node class was not defined properly.	Defined the Node class before using it.
2.	Attribute Error occurred when temp.next was not handled correctly.	Updated temp using temp = temp.next.
3.	Index Error occurred when an invalid index was entered.	Checked whether the index is within the list range.
4.	Value not found occurred when the entered value was not present in the list.	Searched the list and displayed "Value not found."
5.	Empty list error occurred when an operation was performed on an empty list.	Checked self.head is None before performing the operation.

B.AIM: Write a Python program to implement a Double linked list with the above operations.

PROGRAM:

```
class Node: def init(self, data): self.data = data self.prev = None self.next = None
```

```
class DoublyLinkedList: def init(self): self.head = None
```

```
# 1. Create linked list
```

```
def create(self):
```

```
    n = int(input("Enter number of nodes: "))
```

```
    for i in range(n):
```

```
        data = int(input(f"Enter data for node {i + 1}: "))
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next:
```

```
                temp = temp.next
```

```
            temp.next = new_node
```

```
            new_node.prev = temp
```

```
    print("Doubly linked list created successfully.")
```

```
# 2. Insert at beginning
```

```
def insert_beginning(self):
```

```
    data = int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
    if self.head is not None:
```

```
        new_node.next = self.head
```

```
        self.head.prev = new_node
```

```
    self.head = new_node
```

```
    print("Node inserted at beginning.")
```

```
# 3. Insert at end
```

```
def insert_end(self):
```

```
    data = int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next:
```

```
temp = temp.next
temp.next = new_node
new_node.prev = temp
print("Node inserted at end.")
```

4. Insert at specific index

```
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    new_node = Node(data)
    if index == 0:
        if self.head:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
            print("Node inserted.")
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Invalid index.")
            return
        temp = temp.next
    if temp is None:
        print("Invalid index.")
        return
    new_node.next = temp.next
    new_node.prev = temp
    if temp.next:
        temp.next.prev = new_node
    temp.next = new_node
    print("Node inserted.")
```

5. Delete by value

```
def delete_value(self):
    value = int(input("Enter value to delete: "))
    temp = self.head
    while temp and temp.data != value:
        temp = temp.next
    if temp is None:
        print("Value not found.")
    return
```

```
if temp.prev:
    temp.prev.next = temp.next
else:
    self.head = temp.next
```

```
if temp.next:
    temp.next.prev = temp.prev
```

```
print("Node deleted.")
```

6. Delete first node

```
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return
```

```
self.head = self.head.next
```

```
if self.head:
    self.head.prev = None
```

```
print("First node deleted.")
```

7. Delete last node

```
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return
```

```
temp = self.head
```

```
while temp.next:
    temp = temp.next
```

```
if temp.prev:
    temp.prev.next = None
else:
    self.head = None
```

```
print("Last node deleted.")
```

8. Count nodes

```
def count(self):
    count = 0
    temp = self.head

    while temp:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)
```

9. Display / Traverse

```
def display(self):
    if self.head is None:
        print("List is empty.")
        return
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")
```

#Main program

```
dll = DoublyLinkedList()
```

```
while True:
```

```
    print("\n--- DOUBLY LINKED LIST ---")
```

```
    print("1. Create linked list")
```

```
    print("2. Insert at beginning")
```

```
    print("3. Insert at end")
```

```
    print("4. Insert at specific index")
```

```
    print("5. Delete by value")
```

```
    print("6. Delete first node")
```

```
    print("7. Delete last node")
```

```
    print("8. Count number of nodes")
```

```
    print("9. Display / Traverse")
```

```
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()
elif choice == 2:
    dll.insert_beginning()
elif choice == 3:
    dll.insert_end()
elif choice == 4:
    dll.insert_index()
elif choice == 5:
    dll.delete_value()
elif choice == 6:
    dll.delete_first()
elif choice == 7:
    dll.delete_last()
elif choice == 8:
    dll.count()
elif choice == 9:
    dll.display()
elif choice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
```

```
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab5 dll.py =====
```

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 1

Enter number of nodes: 5

Enter data for node 1: 11

Enter data for node 2: 81

Enter data for node 3: 18

Enter data for node 4: 25

Enter data for node 5: 2

Doubly linked list created successfully.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 2

Enter data: 33

Node inserted at beginning.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 3

Enter data: 5

Node inserted at end.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 77

Invalid choice.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 4

Enter index: 3

Enter data: 24

Node inserted.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 5

Enter value to delete: 24

Node deleted.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 5

Enter value to delete: 34

Value not found.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 6

First node deleted.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Enter your choice: 7

Last node deleted.

```
---- DOUBLY LINKED LIST ----
```

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

```

6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
11 <-> 81 <-> 18 <-> 25 <-> 2 <-> None

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.

```

EXPLANATION:

- The Node class creates a node containing **data, previous link, and next link**.
- The DoublyLinkedList class maintains the list using the head pointer.
- The create() function creates the doubly linked list by adding the given nodes.
- The insert_beginning() function adds a **new node at the beginning** and updates the previous link.
- The insert_end() function adds a **new node at the end** of the list.
- The insert_index() function inserts a node at a **specified index** and updates both links.
- The delete_value() function searches for a given value and **removes that node**.
- The delete_first() function removes the **first node** and updates the head.
- The delete_last() function removes the **last node** from the list.
- The count() function counts and displays the **number of nodes**.
- The display() function traverses the list and displays the elements using <->.
- The main program provides a **menu to select and perform different operations**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because the statements were not properly aligned.	Corrected the indentation of all blocks.
2.	NameError occurred because new_node, temp, or other variables were misspelled.	Corrected the variable names and spelling.
3.	AttributeError occurred when prev or next links were not updated correctly.	Updated the prev and next links properly.
4.	IndexError occurred when an invalid index was entered.	Checked whether the index is within the valid range.

C.AIM: Write a Python program to implement a Circular linked list with the above operations.

PROGRAM:

```
class Node:

    def init(self, data):

        self.data = data self.next = None

class CircularLinkedList:

    def init(self):

        self.head = None

# 1. Create linked list
def create(self):
    n = int(input("Enter number of nodes: "))
    for i in range(n):

        data = int(input(f"Enter data for node {i + 1}: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head

    print("Circular linked list created successfully.")

# 2. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
```

```
if self.head is None:
    self.head = new_node
    new_node.next = self.head
else:
    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    new_node.next = self.head
    temp.next = new_node
    self.head = new_node

print("Node inserted at beginning.")
```

3. Insert at end

```
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
```

```
if self.head is None:
    self.head = new_node
    new_node.next = self.head
else:
    temp = self.head
```

```
    while temp.next != self.head:
        temp = temp.next
```

```
    temp.next = new_node
    new_node.next = self.head
```

```
print("Node inserted at end.")
```

4. Insert at specific index

```
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    new_node = Node(data)
```

```
if index == 0:
    if self.head is None:
        self.head = new_node
```

```

        new_node.next = self.head
    else:
        temp = self.head

    while temp.next != self.head:
        temp = temp.next

    new_node.next = self.head
    temp.next = new_node
    self.head = new_node

    print("Node inserted.")
    return

if self.head is None:
    print("Invalid index.")
    return

temp = self.head

for i in range(index - 1):
    temp = temp.next

    if temp == self.head:
        print("Invalid index.")
        return

    new_node.next = temp.next
    temp.next = new_node

    print("Node inserted.")

# 5. Delete by value
def delete_value(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("List is empty.")
        return

    # If head contains the value
    if self.head.data == value:

```

```
if self.head.next == self.head:
    self.head = None
else:
    temp = self.head
```

```
while temp.next != self.head:
    temp = temp.next
```

```
temp.next = self.head.next
self.head = self.head.next
```

```
print("Node deleted.")
return
```

```
temp = self.head
```

```
while temp.next != self.head and temp.next.data != value:
    temp = temp.next
```

```
if temp.next == self.head:
    print("Value not found.")
else:
    temp.next = temp.next.next
    print("Node deleted.")
```

6. Delete first node

```
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return
```

```
if self.head.next == self.head:
    self.head = None
else:
    temp = self.head
```

```
while temp.next != self.head:
    temp = temp.next
```

```
temp.next = self.head.next
self.head = self.head.next
```

```
print("First node deleted.")
```

```
# 7. Delete last node
```

```
def delete_last(self):
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
    if self.head.next == self.head:
```

```
        self.head = None
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next.next != self.head:
```

```
            temp = temp.next
```

```
        temp.next = self.head
```

```
print("Last node deleted.")
```

```
# 8. Count nodes
```

```
def count(self):
```

```
    if self.head is None:
```

```
        print("Number of nodes: 0")
```

```
        return
```

```
    count = 0
```

```
    temp = self.head
```

```
    while True:
```

```
        count += 1
```

```
        temp = temp.next
```

```
    if temp == self.head:
```

```
        break
```

```
    print("Number of nodes:", count)
```

```
# 9. Display / Traverse
```

```
def display(self):
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```

temp = self.head

while True:
    print(temp.data, end=" -> ")
    temp = temp.next

    if temp == self.head:
        break

print("HEAD")
#Main program

c11 = CircularLinkedList()

while True:

    print("\n--- CIRCULAR LINKED LIST ---")

    print("1. Create linked list")

    print("2. Insert at beginning")

    print("3. Insert at end")

    print("4. Insert at specific index")

    print("5. Delete by value")

    print("6. Delete first node")

    print("7. Delete last node")

    print("8. Count number of nodes")

    print("9. Display / Traverse")

    print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    c11.create()
elif choice == 2:
    c11.insert_beginning()
elif choice == 3:

```

```

        cll.insert_end()
    elif choice == 4:
        cll.insert_index()
    elif choice == 5:
        cll.delete_value()
    elif choice == 6:
        cll.delete_first()
    elif choice == 7:
        cll.delete_last()
    elif choice == 8:
        cll.count()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Program exited.")
        break

else:
    print("Invalid choice.")

```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab5 cll.py =====

--- CIRCULAR LINKED LIST ---
1. Create Linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 11
Enter data for node 2: 18
Enter data for node 3: 81
Enter data for node 4: 25
Enter data for node 5: 2
Circular linked list created successfully.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 33
Node inserted at beginning.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes

```

```

10. Exit
Enter your choice: 3
Enter data: 5
Node inserted at end.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 3
Enter data: 7
Node inserted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 46
Value not found.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 33

```

```

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value

```

```

6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 11
Invalid choice.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
>>>

```

EXPLANATION:

- The Node class creates a node containing **data** and a **link to the next node**.
- The CircularLinkedList class maintains the list using the head pointer.

- The create() function creates the circular linked list and connects the **last node back to the head**.
- The insert_beginning() function adds a **new node at the beginning** and updates the circular connection.
- The insert_end() function adds a **new node at the end** and connects it back to the head.
- The insert_index() function inserts a node at a **specified index**.
- The delete_value() function searches for a value and **deletes the corresponding node**.
- The delete_first() function removes the **first node** and updates the head.
- The delete_last() function removes the **last node** while maintaining the circular connection.
- The count() function counts and displays the **total number of nodes**.
- The display() function traverses the list and displays the elements until it reaches the **head again**.
- The main program provides a **menu to perform different circular linked-list operations**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because the statements were not properly aligned.	Corrected the indentation of all code blocks.
2.	NameError occurred because variables like temp or new_node were misspelled.	Corrected the variable names and spelling.
3.	AttributeError occurred when the next link was not updated properly.	Updated the next links correctly to maintain the circular connection.
4.	IndexError occurred when an invalid index was entered.	Checked whether the index is within the valid range.
5.	Value not found occurred when the entered value was not present in the list.	Searched the list and displayed "Value not found."

Week-6

A.AIM: Write a Python program to implement stack operations such as Push, Pop, Peek, Display using an array

PROGRAM:

```
stack = []

# Push operation

def push():

    value = int(input("Enter element: "))

    stack.append(value)

    print("Element pushed.")

# Pop operation

def pop():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Popped element:", stack.pop())

# Peek operation

def peek():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Top element:", stack[-1])

# Display operation

def display():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Stack elements:")
```

```
        for i in range(len(stack) - 1, -1, -1):

            print(stack[i])

# Main program

while True:

    print("\n--- STACK USING ARRAY ---")

    print("1. Push")

    print("2. Pop")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        push()

    elif choice == 2:

        pop()

    elif choice == 3:

        peek()

    elif choice == 4:

        display()

    elif choice == 5:

        print("Program exited.")

        break

    else:

        print("Invalid choice.")
```

OUTPUT:

```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
```

```
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab6.py =====
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 8
Invalid choice.
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element pushed.
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 25
Element pushed.
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element pushed.
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
```

```
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element pushed.
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 15
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 29
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
29
25
2
```

```
--- STACK USING ARRAY ---
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
```

```
>>>
```

EXPLANATION:

- The stack list is used to store the **stack elements**.
- The push() function adds an element to the **top of the stack**.
- The pop() function removes and returns the **top element**.
- The peek() function displays the **top element without removing it**.
- The display() function prints all elements from **top to bottom**.
- len(stack) == 0 checks whether the stack is **empty**.
- The main program provides a **menu** to perform different stack operations.
- Stack follows the **LIFO (Last In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
2.	NameError occurred because stack or a function name was misspelled.	Corrected the variable and function names.
3.	Index Error occurred when trying to access the top element of an empty stack.	Checked whether the stack is empty before accessing an element.
4.	Empty stack error occurred when Pop or Peek was performed without any elements.	Checked <code>len(stack) == 0</code> before performing Pop or Peek.
5.	SyntaxError occurred because of incorrect Python syntax.	Checked and corrected the syntax of the program.

B.AIM: Write a Python program to implement stack operations such as Push, Pop, Peek, Display using a linked list

PROGRAM:

```
class Node:
```

```
    def init(self, data):
```

```
        self.data = data self.next = None
```

```
class Stack:

    def init(self):

        self.top = None

# Push operation
def push(self):
    value = int(input("Enter element: "))

    new_node = Node(value)
    new_node.next = self.top
    self.top = new_node

    print("Element pushed.")

# Pop operation
def pop(self):
    if self.top is None:
        print("Stack is empty.")
    else:
        print("Popped element:", self.top.data)
        self.top = self.top.next

# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty.")
    else:
        print("Top element:", self.top.data)

# Display operation
def display(self):
    if self.top is None:
        print("Stack is empty.")
    else:
        temp = self.top

        print("Stack elements:")
        while temp:
            print(temp.data)
            temp = temp.next
```

```
#Main program

stack = Stack()

while True:

    print("\n--- STACK USING LINKED LIST ---")

    print("1. Push")

    print("2. Pop")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        stack.push()

    elif choice == 2:
        stack.pop()

    elif choice == 3:
        stack.peak()

    elif choice == 4:
        stack.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab6(b) ll.py =====

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element pushed.

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 25
Element pushed.

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element pushed.

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element pushed.

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek

```

```

4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element pushed.

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 15

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 29

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
29
25
2

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
>>>

```

EXPLANATION:

- The Node class creates a node containing **data and a link to the next node**.
- The Stack class uses top to keep track of the **top element**.
- The push() function adds a new node at the **top of the stack**.
- The pop() function removes the **top node** from the stack.
- The peek() function displays the **top element without removing it**.
- The display() function traverses the linked list and displays all elements.
- self.top is None checks whether the stack is **empty**.
- The main program provides a **menu** for performing stack operations.
- The stack follows the **LIFO (Last In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	NameError occurred because top, temp, or new_node was misspelled.	Corrected the variable names and their spelling.
2.	AttributeError occurred when next was accessed incorrectly.	Correctly used self.top and temp.next.
3.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
4.	Empty stack error occurred when Pop or Peek was performed without any elements.	Checked self.top is None before performing the operation.
5.	IndentationError occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.

Week-7

A.AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array

PROGRAM:

```
queue = []
```

```
# Enqueue operation
```

```
def enqueue():

    value = int(input("Enter element: "))

    queue.append(value)

    print("Element inserted.")

# Dequeue operation

def dequeue():

    if len(queue) == 0:

        print("Queue is empty.")

    else:

        print("Deleted element:", queue.pop(0))

# Peek operation

def peek():

    if len(queue) == 0:

        print("Queue is empty.")

    else:

        print("Front element:", queue[0])

# Display operation

def display():

    if len(queue) == 0:

        print("Queue is empty.")

    else:

        print("Queue elements:")

        for element in queue:

            print(element, end=" ")

        print()
```

```
# Main program

while True:

    print("\n--- QUEUE USING ARRAY ---")

    print("1. Enqueue")

    print("2. Dequeue")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        enqueue()

    elif choice == 2:

        dequeue()

    elif choice == 3:

        peek()

    elif choice == 4:

        display()

    elif choice == 5:

        print("Program exited.")

        break

    else:

        print("Invalid choice.")
```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab7(a) array.py =====
--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 25
Element inserted.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element inserted.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element inserted.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek

```

```

4. Display
5. Exit
Enter your choice: 2
Deleted element: 2

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 25

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
25 29 15

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
>>>

```

EXPLANATION:

- The queue list is used to store the **queue elements**.
- The enqueue() function adds an element at the **rear** of the queue.
- The dequeue() function removes an element from the **front** of the queue.
- The peek() function displays the **front element** without removing it.
- The display() function prints all the elements in the queue.
- `len(queue) == 0` checks whether the queue is **empty**.
- The main program provides a **menu** to select different queue operations.
- Queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.
2.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
3.	NameError occurred because queue or a function name was misspelled.	NameError occurred because queue or a function name was misspelled.
4.	IndexError occurred when trying to access an element from an empty queue.	Checked whether the queue is empty before accessing elements.
5.	Empty queue error occurred when Dequeue or Peek was performed without elements.	Checked <code>len(queue) == 0</code> before performing the operation.

B.AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Linked list data.

PROGRAM:

```
class Node:
```

```
    def init(self, data):
```

```
        self.data = data self.next = None
```

```
class Queue:
```

```
    def init(self):
```

```
        self.front = None self.rear = None
```

```
# Enqueue operation
```

```
def enqueue(self):
```

```
    value = int(input("Enter element: "))
```

```
    new_node = Node(value)
```

```
    if self.rear is None:
```

```
        self.front = self.rear = new_node
```

```
    else:
```

```
        self.rear.next = new_node
```

```
        self.rear = new_node
```

```

    print("Element inserted.")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Deleted element:", self.front.data)
        self.front = self.front.next

    if self.front is None:
        self.rear = None

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        temp = self.front

        print("Queue elements:")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()

#Main program
queue = Queue()

while True:

    print("\n--- QUEUE USING LINKED LIST ---")

    print("1. Enqueue")

```

```

print("2. Dequeue")

print("3. Peek")

print("4. Display")

print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    queue.enqueue()
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()
elif choice == 5:
    print("Program exited.")
    break
else:
    print("Invalid choice.")

```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab7(b) ll.py =====
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 8
Invalid choice.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element inserted.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element inserted.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display

```

```

4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element inserted.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 2

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 15

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
15 29 15

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
>>>

```

EXPLANATION:

- The Node class creates a node containing **data and a link to the next node**.
- The Queue class uses front and rear to maintain the queue.
- The enqueue() function adds a new node at the **rear**.
- The dequeue() function removes a node from the **front**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the linked list and displays all elements.
- The queue follows the **FIFO (First In, First Out)** principle.
- The main program provides a **menu-driven approach** for performing the operations.

ERROR TABLE:

S.no.	Errors	Rectification
1.	NameError occurred because front, rear, or new_node was misspelled.	Corrected the variable names and their spelling.
2.	AttributeError occurred when next was accessed incorrectly.	Used self.front, self.rear, and temp.next correctly.
3.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
4.	Empty queue error occurred when Dequeue or Peek was performed without elements.	Checked self.front is None before performing the operation.
5.	SyntaxError occurred due to incorrect Python syntax.	Checked and corrected the syntax of the program.

C.AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue array structures

PROGRAM:

```
class CircularQueue:

    def init(self, size):

        self.size = size self.queue = [None] * size self.front = -1 self.rear = -1

# Enqueue
def enqueue(self):
    value = int(input("Enter element: "))

    if (self.rear + 1) % self.size == self.front:
        print("Queue is full.")
        return

    if self.front == -1:
        self.front = 0

    self.rear = (self.rear + 1) % self.size
    self.queue[self.rear] = value

    print("Element inserted.")
```

```

# Dequeue
def dequeue(self):
    if self.front == -1:
        print("Queue is empty.")
        return

    print("Deleted element:", self.queue[self.front])

    if self.front == self.rear:
        self.front = -1
        self.rear = -1
    else:
        self.front = (self.front + 1) % self.size

# Peek
def peek(self):
    if self.front == -1:
        print("Queue is empty.")
    else:
        print("Front element:", self.queue[self.front])

# Display
def display(self):
    if self.front == -1:
        print("Queue is empty.")
        return

    print("Queue elements:")
    i = self.front

    while True:
        print(self.queue[i], end=" ")

        if i == self.rear:
            break

        i = (i + 1) % self.size

    print()

#Main program

size = int(input("Enter queue size: "))

queue = CircularQueue(size)

```

```
while True:
```

```
    print("\n--- CIRCULAR QUEUE ---")
```

```
    print("1. Enqueue")
```

```
    print("2. Dequeue")
```

```
    print("3. Peek")
```

```
    print("4. Display")
```

```
    print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
```

```
    queue.enqueue()
```

```
elif choice == 2:
```

```
    queue.dequeue()
```

```
elif choice == 3:
```

```
    queue.peek()
```

```
elif choice == 4:
```

```
    queue.display()
```

```
elif choice == 5:
```

```
    print("Program exited.")
```

```
    break
```

```
else:
```

```
    print("Invalid choice.")
```

OUTPUT:

```
>>> Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab7(c) cll.py =====
Enter queue size: 5

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted.

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 25
Element inserted.

...

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element inserted.

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element inserted.

--- CIRCULAR QUEUE ---
1. Enqueue
```

```

2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 2

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 25

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
25 29 15

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 10
Invalid choice.

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
>>>

```

EXPLANATION:

- The CircularQueue class creates a **queue using an array**.
- front represents the position of the **first element**, while rear represents the **last element**.
- enqueue() adds a new element at the **rear**.
- dequeue() removes an element from the **front**.
- peek() displays the **front element** without deleting it.
- display() prints all the elements from **front to rear**.
- The % size operation makes the rear and front **wrap around the array**.
- The main program provides a **menu-driven interface**.
- The queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because the statements were not properly aligned.	Corrected the indentation of all code blocks.
2.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
3.	NameError occurred because front, rear, or queue was misspelled.	Corrected the variable names and spelling.
4.	IndexError occurred when an invalid array position was accessed.	Used proper circular indexing with % size.
5.	Queue full error occurred when an element was inserted into a full queue.	Checked the full condition before enqueueing.

D.AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue Linked list structures.

PROGRAM:

```
class Node:

    def init(self, data):

        self.data = data self.next = None

class CircularQueue:

    def init(self):

        self.front = None self.rear = None

# Enqueue
def enqueue(self):

    value = int(input("Enter element: "))
    new_node = Node(value)

    if self.front is None:
        self.front = self.rear = new_node
        new_node.next = self.front
    else:
        new_node.next = self.front
        self.rear.next = new_node
        self.rear = new_node

    print("Element inserted.")

# Dequeue
def dequeue(self):
    if self.front is None:
        print("Queue is empty.")
        return

    print("Deleted element:", self.front.data)

    if self.front == self.rear:
        self.front = None
        self.rear = None
    else:
```

```

        self.front = self.front.next
        self.rear.next = self.front

# Peek
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return

    temp = self.front

    print("Queue elements:")

    while True:
        print(temp.data, end=" ")

        temp = temp.next

        if temp == self.front:
            break

    print()

#Main program
queue = CircularQueue()

while True:

    print("\n--- CIRCULAR QUEUE USING LINKED LIST ---")

    print("1. Enqueue")

    print("2. Dequeue")

    print("3. Peek")

```

```

print("4. Display")

print("5. Exit")

choice = int(input("Enter your choice: "))
if choice == 1:
    queue.enqueue()
elif choice == 2:
    queue.dequeue()
elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()
elif choice == 5:
    print("Program exited.")
    break
else:
    print("Invalid choice.")

```

OUTPUT:

```

Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab7(d) cll ll.py =====

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 25
Element inserted.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 29
Element inserted.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 15
Element inserted.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek

```

```

4. Display
5. Exit
Enter your choice: 2
Deleted element: 2

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 25

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
25 29 15

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 10
Invalid choice.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
>>>

```

EXPLANATION:

- The Node class creates a node containing **data and a next link**.
- The CircularQueue class maintains the queue using front and rear.
- The enqueue() function adds a node at the **rear** and connects it back to the front.
- The dequeue() function removes the **front node**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the circular linked list until it reaches the **front again**.
- front and rear are set to None when the queue becomes empty.
- The main program provides a **menu-driven approach**.
- The circular queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	NameError occurred because front, rear, or new_node was misspelled.	Corrected the variable names and spelling.
2.	AttributeError occurred when the next link was not accessed correctly.	Used self.front, self.rear, and temp.next correctly.
3.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
4.	Empty queue error occurred when Dequeue or Peek was performed on an empty queue.	Checked self.front is None before performing the operation.
5.	Circular link error occurred when the rear node was not connected back to the front node.	Updated self.rear.next = self.front after insertion and deletion.